OASIS WEB6 · WHITEPAPER

HOLONIC BRAID & FAHRN

FAHRN – FRACTAL ADAPTIVE HOLONIC REASONING NETWORK

Fractal holonic shared memory, collective intelligence and the Meta-Orchestration Intelligence Layer – a principled path to AGI modelled after nature, unity consciousness and the infinite intelligence of the universe.

WEB6 v2.0 | MAJOR ARCHITECTURE UPGRADE

VERSION 2.0 · JULY 2026 · NEXTGEN SOFTWARE UK LTD · OASIS WEB6

TABLE OF CONTENTS

01 · ABSTRACT

Abstract

This whitepaper introduces two interconnected architectures that together form the intelligence foundation of **OASIS WEB6**, built upon the BRAID framework (Amçalar & Cinar, arXiv:2512.15959) developed within the OpenSERV platform:

Holonic BRAID extends the OpenSERV BRAID (Bounded Reasoning for Autonomous Inference and Decisions) framework with a fractal, hierarchical shared memory system modelled after the holonic structures found in nature. Every AI session produces a memory holon. That holon belongs to a parent agent holon, which belongs to a user holon, which belongs to groups, which belongs to geographic communities, all the way up to the Earth holon. At each boundary, user-configurable membrane rules govern exactly what memory propagates upward. The result is genuine collective intelligence — built bottom-up from billions of individual interactions — rather than the fragmented, siloed AI memory that exists today.

FAHRN (the Fractal Adaptive Holonic Reasoning Network) is a Meta-Orchestration Intelligence Layer built on top of Holonic Braid. A controller agent manages a network of specialised AI reasoning agents (GPT-5, Claude, Grok, Gemini and others), each carrying live performance metadata scored by problem category and speed. The controller dispatches problems in one of three modes — serial (cost-optimised), parallel (accuracy-optimised) or decomposed (complex problems) — and assembles the best execution plan from the results. All outcomes feed back into the Holonic Braid memory hierarchy, continuously improving future routing decisions.

Together these systems represent a fundamentally different approach to machine intelligence: one modelled after the infinite intelligence of nature and the universe, built through *unity consciousness* rather than the fragmented separation consciousness that currently defines AI, society and human civilisation.

Core thesis: Collective intelligence cannot be engineered top-down. It must emerge bottom-up, exactly as it does in nature — through billions of individual interactions propagating upward through a fractal holonic hierarchy, unified by shared memory and governed by self-chosen membrane rules at every boundary.

The BRAID Framework — OpenSERV Foundation

BRAID — Bounded Reasoning for Autonomous Inference and Decisions — is a multi-agent reasoning framework introduced by Amçalar & Cinar (arXiv:2512.15959) and implemented within the OpenSERV platform. It is the technical foundation upon which the OASIS Holonic BRAID architecture is built.

The core observation driving BRAID is that generating a reasoning graph for a task type is expensive but reusable — while executing that graph against a specific task instance is cheap. By separating these two concerns into a two-stage protocol, BRAID achieves dramatic Performance Per Dollar (PPD) gains over conventional single-model approaches.

> **Key result:** BRAID delivers a **74× PPD gain** on GSM-Hard mathematical reasoning benchmarks (gpt-4.1 Generator → gpt-5-nano-minimal Solver) and a **30× gain** on procedural tasks, compared to a GPT-5-medium full-reasoning baseline. Accuracy simultaneously improves: GSM-Hard goes from **94% → 98%**.

2.1 — Two-Stage Protocol: Generator + Solver

BRAID separates reasoning into two distinct roles:

GENERATOR STAGE

A **high-tier model** (e.g. gpt-4.1) analyses a task type τ and constructs a structured **Mermaid reasoning graph** — a step-by-step execution blueprint that captures the optimal reasoning strategy for that class of problem. This graph is generated *once per task type*, not once per task.

SOLVER STAGE

A **low-tier model** (e.g. gpt-5-nano-minimal) receives the pre-generated Mermaid graph and executes it against the specific task instance. Because the reasoning strategy is already encoded in the graph, the solver needs far less compute — and produces results competitive with or superior to the high-tier model running alone.

Two-stage flow:

```
Task τ instance arrives
    ↓
[LOOKUP] Does graph library contain a reasoning graph for task type τ?
    ├── YES → retrieve graph from library holon (zero generation cost)
    └── NO → GENERATOR (gpt-4.1) creates Mermaid graph for τ
                → store new graph in library holon
    ↓
SOLVER (gpt-5-nano-minimal) executes graph against τ instance
    ↓
Result + performance data → Session Holon → Holonic Braid hierarchy
```

2.2 — Performance Per Dollar (PPD) & Cost Equations

PPD is the primary metric used to evaluate BRAID efficiency. It measures the accuracy gain achieved per unit of inference cost, relative to a single-model baseline.

COST MODEL — HOLONIC BRAID AT SCALE

Let:

- **Q** = number of unique task types in the library
- **T** = total number of task instances to solve
- C_{gen} = cost of one Generator call (high-tier model, e.g. gpt-4.1)
- C_{solve} = cost of one Solver call (low-tier model, e.g. gpt-5-nano-minimal)
- C_{GPT5} = cost of one full GPT-5-medium call (baseline comparator)

Total cost — Holonic BRAID:

$$Cost = Q \cdot C_{gen} + T \cdot C_{solve}$$

Performance Per Dollar (PPD):

$$PPD = (C_{GPT5} \cdot T) \,/\, (Q \cdot C_{gen} + T \cdot C_{solve})$$

As T grows large relative to Q (many tasks, few unique task types), the $Q \cdot C_{gen}$ term becomes negligible and PPD approaches its maximum: $\mathbf{C_{GPT5} / C_{solve}}$.

At T = 10,000 tasks and Q = 50 unique task types: the 50 generator calls are a one-time cost shared across 10,000 solver calls. The generator cost is 0.5% of the total. The PPD on GSM-Hard reaches **74×** the

baseline.

WHY BRAID WITHOUT SHARING COLLAPSES AT SCALE

Standard BRAID (without the Holonic shared library) requires each agent instance to generate its own reasoning graph for each task type it encounters. When many agents independently process similar tasks, the generator cost is paid repeatedly — once per agent per task type rather than once globally.

> **BRAID no-share PPD at scale:**
>
> $$\text{Cost}_{\text{no-share}} = T \cdot C_{\text{gen}} + T \cdot C_{\text{solve}} = T \cdot (C_{\text{gen}} + C_{\text{solve}})$$
>
> As $T \to \infty$, $\text{PPD}_{\text{no-share}} \to C_{\text{GPT5}} / (C_{\text{gen}} + C_{\text{solve}}) \approx \mathbf{1.5\times}$
>
> The savings collapse because the expensive generator is called for every task rather than once per task type. This is the core problem that Holonic BRAID solves.

2.3 — Benchmark Results

BRAID was evaluated against three standard reasoning benchmarks. Holonic BRAID extends these gains — maintaining them at scale where standard BRAID degrades.

BENCHMARK	BASELINE (GPT-5-MEDIUM)	BRAID / HOLONIC BRAID	PPD GAIN
GSM-Hard (math reasoning)	94% accuracy	98% accuracy	74×
SCALE MultiChallenge	23.9%	45.2%	significant
AdvancedIF (instruction following)	baseline	substantial gain	measured
Procedural tasks	1× (baseline)	equivalent accuracy	30×

BRAID NO-SHARE VS. HOLONIC BRAID — COMPARISON

APPROACH	PPD (LOW T)	PPD (HIGH T)	SCALE BEHAVIOUR
GPT-5-medium (baseline)	1.0×	1.0×	Stable, expensive
BRAID (no sharing)	74× / 30×	~1.5×	Collapses as T grows
Holonic BRAID	74× / 30×	74×+ sustained	Improves with scale

2.4 — The Scale Problem: Why Sharing Matters

The BRAID paper demonstrates impressive PPD gains in single-agent settings. However, real-world deployment involves many agents handling similar tasks independently. Without a shared graph library, each agent regenerates reasoning graphs for task types already solved by thousands of other agents. The generator cost, amortised across an entire platform, is astronomical — and the PPD gains evaporate.

This is the problem that Holonic BRAID was designed to solve. By storing reasoning graphs as holons in a shared, replicated library — accessible to all agents with appropriate permissions — every task type that any agent has ever reasoned about is available to all future agents at zero generation cost. The generator is called once globally; the solver is called once per task instance. This is the only way to sustain BRAID's PPD gains at platform scale.

The OASIS insight: BRAID solves the per-agent reasoning cost problem. Holonic BRAID solves the cross-agent, cross-session, cross-scale sharing problem. Together they form a system where PPD *improves* with scale rather than collapsing — because every new task type solved anywhere enriches the shared library for everyone.

Holonic BRAID — The OASIS Extension

Holonic BRAID extends the OpenSERV BRAID framework with two foundational additions: a **shared reasoning graph library** stored as a holonic data structure, and **cross-chain persistence** across multiple storage backends via the OASIS COSMIC ORM. These additions transform BRAID from a per-agent optimisation into a platform-scale collective intelligence system.

3.1 — The Shared Graph Library as a Holon

In Holonic BRAID, the reasoning graph library is not a flat database — it is itself a **holon**. The library holon is a parent holon whose children are individual graph holons, one per task type τ.

Library holon structure:

```
LIBRARY_HOLON {
  Id: globally-unique GUID
  Children: [ GRAPH_HOLON(τ₁), GRAPH_HOLON(τ₂), … GRAPH_HOLON(τₙ) ]
  ProviderUniqueStorageKey: { MongoDB: "…", Solana: "…", IPFS: "…" }
}

GRAPH_HOLON(τ) {
  Id: globally-unique GUID
  Parent: LIBRARY_HOLON.Id
  Metadata: { task_type: τ, created_by: agent_id, created_at: timestamp,
              usage_count: N, avg_solver_accuracy: 0.97 }
  MermaidGraph: "graph TD; A[Parse input]→B[Classify]→…"
  ProviderUniqueStorageKey: { MongoDB: "…", Solana: "…", IPFS: "…" }
}
```

The **lookup-or-create** pattern governs every task: when an agent encounters task type τ, it queries the library holon first. If a graph exists, it is retrieved and passed directly to the Solver. If no graph exists, the Generator is invoked and the resulting graph is stored as a new child holon of the library — immediately available to all other agents.

LOOKUP-OR-CREATE

Every agent checks the shared library before invoking the Generator. As the library grows, Generator calls become increasingly rare – amortised across the full agent population.

IMPROVING ACCURACY

Graph holons accumulate quality metadata over time: usage count, average solver accuracy, user satisfaction. The FAHRN preferentially selects higher-quality graphs, and graphs can be regenerated when accuracy degrades below threshold.

GLOBALLY SHARED

Any agent with read access to the library holon benefits from graphs created by any other agent. A reasoning graph generated by one user's session is instantly available to millions of other sessions – at zero marginal cost.

MEMBRANE GOVERNED

Library access is governed by membrane rules. Public graphs are globally readable. Private or proprietary graphs can be scoped to a user, group or organisation – with the same per-field granularity as all other holons in the system.

VERSIONED GRAPHS

Each update to a reasoning graph creates a new versioned holon child – previous versions remain queryable for reproducibility and rollback. When two agents independently produce graphs for the same task type, the FAHRN compares both and selects or merges the superior version.

ZERO LATENCY RETRIEVAL

Graph lookups hit the COSMIC ORM cache layer first – in-memory for hot task types, MongoDB for warm, IPFS for archival. Most common task types resolve in under 2ms, meaning the Solver stage begins almost immediately without waiting for any Generator call.

3.2 – Cross-Chain Persistence

Holons – including reasoning graph holons – are stored via the **OASIS COSMIC ORM**, the universal data abstraction layer that spans 40+ storage providers. Each holon carries a `ProviderUniqueStorageKey` per configured backend, allowing the same holon to be read from or written to any provider transparently.

MONGODB

Primary fast-access store for reasoning graph holons. Rich query support for task type lookup, metadata filtering and graph retrieval. Suitable for low-latency live agent dispatch.

SOLANA

Blockchain persistence layer for immutable graph provenance. When a reasoning graph is published to the shared library, its hash and authorship are anchored on-chain — providing tamper-evident proof of origin and creation time.

IPFS

Decentralised content-addressed storage for censorship-resistant, permanent graph availability. Mermaid graph content is stored on IPFS; the CID (content identifier) is recorded in the holon and anchored on Solana.

ProviderUniqueStorageKey: Each holon stores a map of backend → storage key. The COSMIC ORM resolves reads and writes to the configured provider transparently. An agent does not need to know whether a graph is coming from MongoDB, IPFS or Solana — the same holon Id returns the same graph from any backend. Failover between providers is automatic.

CONSISTENCY & ACCURACY MECHANISMS

MECHANISM	HOW IT WORKS
Graph versioning	Each update creates a new graph holon version; previous versions remain queryable for reproducibility
Accuracy threshold regeneration	If average solver accuracy on a graph drops below a configurable threshold, the Generator is reinvoked to produce an improved replacement
Conflict resolution	When two agents independently generate graphs for the same task type T, the FAHRN compares both and selects or merges the superior version, then updates the library holon
Membrane-gated writes	Agents can only write new graphs to library holons they have write permission for; public library writes are curated to prevent pollution

MECHANISM	HOW IT WORKS
On-chain anchoring	Solana anchors provide tamper-evident immutability – any graph modification produces a new chain record, making the full audit trail available

The holonic memory system continues below into the full hierarchy that underpins the rest of the OASIS WEB6 intelligence layer – including session holons, user holons, geographic holons and the Earth holon.

3.3 — What Is a Holon?

The term *holonic* derives from philosopher Arthur Koestler's concept of the **holon** – something that is simultaneously a whole in itself and a part of a larger whole. This is the fundamental structure of nature: atoms are wholes that are parts of molecules, which are wholes that are parts of cells, which are wholes that are parts of organs, and so on through every scale of existence.

Holonic Braid applies this same fractal logic to AI memory, knowledge and intelligence. Rather than isolated, amnesiac AI sessions that forget everything between conversations – or monolithic centralised models that aggregate data without consent or nuance – Holonic Braid creates a living, hierarchical, consent-governed memory fabric that mirrors how intelligence actually works in nature.

2.1 — What Is a Holon?

In the context of Holonic Braid, a **holon** is a structured memory unit that:

- Contains its own internal state, knowledge and context
- Has a defined boundary – the *membrane* – through which information can pass in or out
- Is simultaneously a complete unit and a component of a larger parent holon
- Can have multiple child holons and belong to multiple parent holons simultaneously
- Participates in bidirectional information flow: children propagate upward, parents contextualise downward

3.4 — The Session Holon

Every AI conversation or task execution produces a **session holon**. This is the atomic unit of the Holonic Braid system. A session holon captures:

CONVERSATION MEMORY

The full context of the session — messages, reasoning steps, conclusions and outcomes — structured as a queryable knowledge object.

PERFORMANCE METADATA

How well did this agent perform in this session? What problem categories were addressed? Speed, accuracy and user satisfaction signals.

RELATIONAL LINKS

Connections to related session holons, referenced knowledge holons, and the parent agent holon — forming a queryable knowledge graph.

MEMBRANE CONFIGURATION

Which parts of this session propagate to the parent agent holon? The user defines this per session, per field, with granular control.

SEMANTIC TAGGING

Every session holon is tagged with inferred topic categories, sentiment, domain and quality signals. Tags drive membrane filtering — "propagate only professional-topic memories to my work group holon" — and enable cross-session semantic search without exposing raw content.

CROSS-PROVIDER SYNC

Session holons are persisted via COSMIC ORM across all configured providers simultaneously — MongoDB for fast retrieval, Solana for immutable provenance, IPFS for distributed backup. A single write operation fans out to every backend; reads resolve from the fastest available source.

3.5 — The Full Holonic Hierarchy

Session holons are the leaves of a vast fractal tree. Each level is a whole in itself and a part of something larger:

🌍 EARTH HOLON

Universal planetary collective intelligence. The sum of all human knowledge, experience and intention as expressed through every AI interaction on Earth. The omega point of the hierarchy.

🌐 CONTINENT HOLONS

Each continent forms a collective intelligence holon — aggregating the shared memory of every country within it, filtered by continental membrane rules.

🏳️ COUNTRY HOLONS

National collective intelligence. Policies, culture, shared knowledge and collective experience at the country level.

🗺️ COUNTY / REGION HOLONS

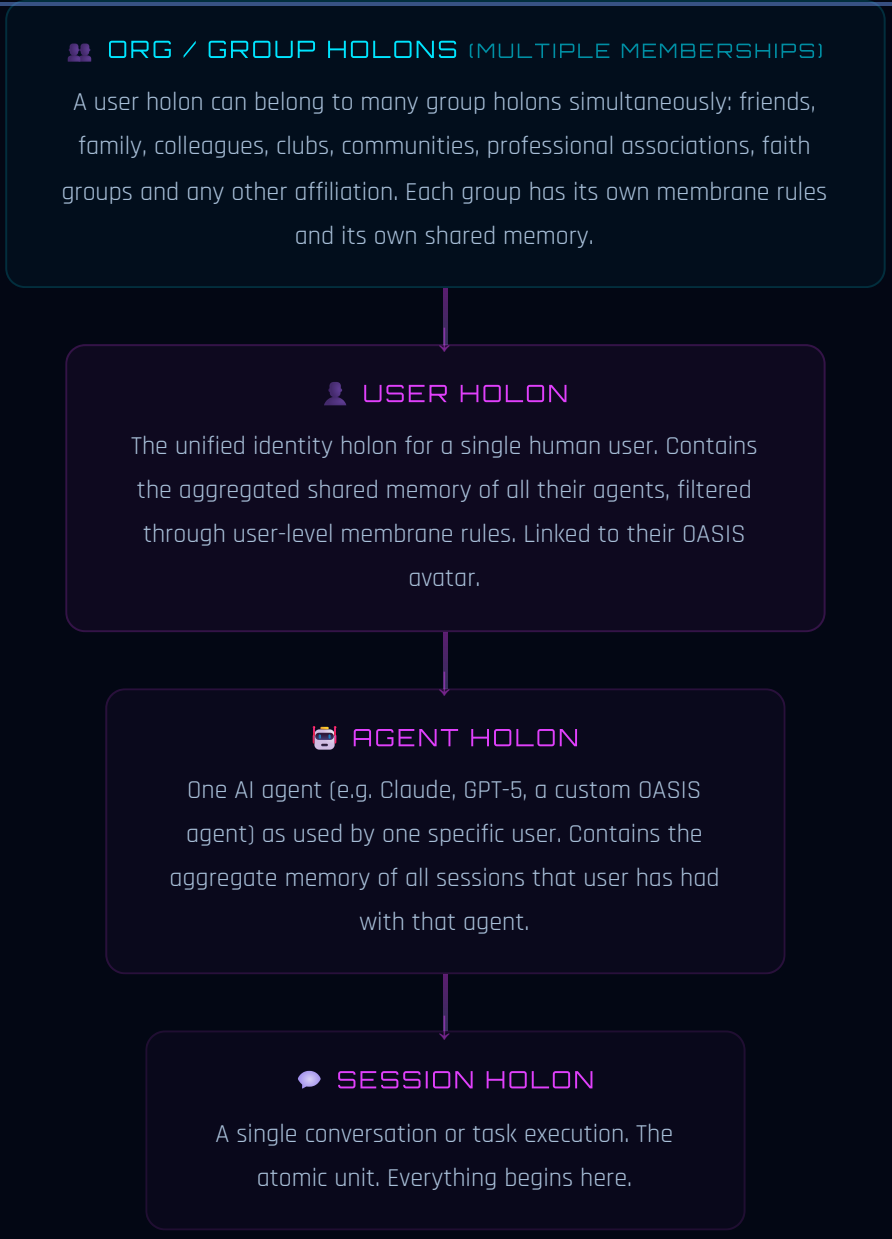
Regional collective intelligence. Local government, community initiatives and shared regional context.

🌆 CITY / TOWN HOLONS

Urban collective intelligence. City services, local businesses, cultural institutions and community knowledge.

🏘️ DISTRICT / AREA HOLONS → NEIGHBOURHOOD HOLONS

Hyperlocal intelligence. The knowledge and shared memory of people in close geographic proximity — enabling truly local-aware AI.



> **Key insight:** The hierarchy is fractal — the same holon structure repeats at every scale. A neighbourhood holon has the same architecture as a session holon: internal state, membrane, parent and children. The pattern is self-similar from the smallest conversation to the Earth itself.

3.6 — Membrane Rules & Privacy

The **membrane** is the most critical component of Holonic Braid. It is the governed boundary through which information passes between holons. Without membranes, you have surveillance. Without membranes, collective intelligence becomes collective exposure. Membranes make the system safe, ethical and genuinely empowering.

At every level of the hierarchy, membrane rules define:

WHAT PROPAGATES

Which fields, topics or knowledge categories from this holon are allowed to flow upward to the parent. Per-field granularity – identical to the OASIS WEB4 field-level data control system.

WHAT IS RETAINED

What parts of the session are persisted locally in the agent/user holon vs. discarded after the session ends. The user can choose ephemeral, session-scoped or permanent retention per topic.

WHO CAN READ

Which parent holons, sibling holons or external agents are permitted to query memory from this holon. Read access is separate from write/propagation access.

TRIGGER CONDITIONS

Rules can be conditional: "propagate this memory to my work group holon only if the topic is tagged as professional" or "share with my neighbourhood holon only anonymised aggregate patterns."

REVOCATION & AUDIT

Membrane permissions are fully revocable at any time. A revocation event propagates downward through all child holons, removing previously-shared data from parent aggregates. Every propagation event is logged as an immutable audit holon – a complete lineage trail of what was shared, when, and with whom.

CROSS-BOUNDARY ANONYMISATION

When memory crosses from a user holon to a group or geographic holon, a configurable anonymisation pipeline strips or generalises PII before the data enters the shared aggregate. Raw personal data never leaves the user holon – only the derived, consented pattern does.

> **Privacy by design:** Nothing propagates without explicit permission. The default is private. The user adds propagation permissions; they are never assumed. This is the inverse of today's data economy.

3.7 – Collective Intelligence Formation

As lower-level holons propagate permitted memory upward through their membrane rules, **genuine collective intelligence forms at every level**. This is not data aggregation or model fine-tuning in the traditional sense – it is a living, continuously updated shared knowledge fabric.

Consider the chain: a million individual session holons (each recording a conversation about local weather, traffic, events) propagate permitted patterns upward → neighbourhood holons develop hyperlocal awareness → city holons develop urban intelligence → country holons develop national knowledge → the Earth holon develops planetary awareness — all without any individual session being exposed beyond what its owner chose.

This mirrors **how intelligence works in nature**: individual neurons fire, patterns form in neural clusters, circuits develop in brain regions, faculties emerge in the whole brain, consciousness arises in the whole organism. No single neuron contains consciousness — and yet consciousness is undeniably real. Holonic Braid creates the same emergent quality in AI.

3.8 — External Memory Provider Integration

The fractal holon hierarchy is the primary memory substrate of Holonic Braid, but the platform is designed to be **memory-provider-neutral**. Developers who already run Mem0, Zep, Letta/MemGPT, LangMem, Graphiti or Redis Vector Memory can plug these directly into the same pipeline — they sit alongside the holon hierarchy rather than replacing it.

UNIFIED MEMORY INTERFACE

A `MemoryProviderManager` abstract interface normalises search, add and delete across every external memory provider — the same way `AIProviderManager` normalises completions. Providers are configured in `OASIS_DNA.json` and activated at runtime with no code changes.

CONTEXT INJECTION PIPELINE

Before any FAHRN dispatch or completion call, the pipeline queries each configured external memory provider for semantically relevant past memories, injects the top-K results into the system context alongside Holonic BRAID graph data. The AI receives the richest possible grounding — from both the platform's own fractal memory and the developer's existing memory infrastructure.

BIDIRECTIONAL SYNC

Memories written during a session flow back into external providers as well as into the Holonic BRAID hierarchy — keeping Mem0, Zep or LangMem in sync with the platform's own holon store. Developers get a single authoritative memory substrate without abandoning their existing infrastructure investment.

PROVIDER	STRENGTH	ENV VAR
Mem0	Auto-managed per-user/session/agent memory; REST API	`MEM0_API_KEY`
Zep	Graph-based long-term memory; entity extraction; dialog history	`ZEP_API_KEY`
Letta / MemGPT	Stateful agent memory with self-editing context windows	`LETTA_API_KEY`
LangMem	LangGraph-native; cloud REST or in-process	`LANGMEM_API_KEY`
Graphiti	Temporal knowledge graph; episode-based memory; entity timelines	`GRAPHITI_BASE_URL`
Redis Vector	Ultra-fast in-process vector search; Redis Stack + RediSearch	`REDIS_URL`

This positions Holonic Braid not as a competing memory system but as a **meta-memory orchestrator** — one that can draw from multiple specialised providers while maintaining the fractal hierarchy's unique property of consent-governed upward propagation to collective intelligence.

FAHRN — Fractal Adaptive Holonic Reasoning Network

The FAHRN is an upgrade layer built on top of Holonic Braid. Where Holonic Braid provides the *memory fabric* — the persistent, hierarchical substrate of intelligence — the FAHRN provides the *active intelligence*: a system for solving hard problems using the right agent, at the right time, in the right configuration.

> **The core insight:** No single AI model is best at everything. GPT-5 may outperform Claude on pure deductive reasoning tasks; Claude may outperform Grok on long-form analysis; Grok may outperform both on real-time information retrieval. Rather than pick one and accept its weaknesses, the FAHRN routes each problem to its optimal solver — and learns from every outcome.

4.1 — The Controller Agent

The **controller agent** is the meta-level orchestrator of the FAHRN. It sits above the pool of reasoning agents and is responsible for:

- **Problem classification** — analysing an incoming problem and assigning it to one or more category tags (reasoning, code generation, real-time search, mathematical proof, creative writing, data analysis, etc.)
- **Mode selection** — choosing serial, parallel or decomposed dispatch mode based on problem complexity, time constraints and cost budget
- **Agent selection** — querying the agent scoring metadata store and ranking available agents by their combined category score + speed score for this problem
- **Dispatch and monitoring** — sending the problem to the selected agent(s) and monitoring execution for timeout, logic loops or stall conditions
- **Fallback promotion** — if an agent stalls, automatically promoting the next highest-ranked agent for that category
- **Plan assembly** — in parallel and decomposed modes, comparing or merging the Mermaid execution diagrams returned by each agent into a single optimal plan
- **Outcome recording** — writing results, scores and performance signals back to the Holonic Braid agent holons for future routing improvement

4.2 — Agent Scoring & Metadata

Every agent in the FAHRN carries a live **scoring metadata object**. This is not static — it updates continuously from real outcomes, stored durably in the Holonic Braid memory layer so that the collective learning of all users' interactions informs future routing (subject to membrane rules).

SCORING DIMENSIONS

GPT-5

Reasoning ★★★★★
Math & Logic ★★★★★
Code Gen ★★★★☆
Speed ★★★★☆
Real-time ★★☆☆☆

CLAUDE

Analysis ★★★★★
Long-form ★★★★★
Writing ★★★★★
Speed ★★★★☆
Safety ★★★★★

GROK

Real-time ★★★★★
Search ★★★★★
Speed ★★★★★
Humour ★★★★★
X-data ★★★★★

GEMINI

Multimodal ★★★★★
Google data ★★★★★
Code ★★★★☆
Speed ★★★★☆
Vision ★★★★★

LLAMA / LOCAL

Private ★★★★★
Cost ★★★★★
Custom ★★★★★
Offline ★★★★★
Speed ★★★☆☆

DEEPSEEK / MISTRAL

Reasoning ★★★★★
Code ★★★★★
Cost ★★★★★
Math ★★★★★
Speed ★★★★☆

AWS / AZURE

Enterprise ★★★★★
Scale ★★★★★
Compliance ★★★★★
Vision ★★★★☆
Cost ★★★☆☆

+

CUSTOM / FUTURE

Any provider
via WEB6 API
Scores populated
from live outcomes

Score schema example:

```
{ "agent": "claude-opus-4-8", "categories": { "analysis": 0.94, "code": 0.87, "reasoning": 0.91, "writing": 0.96, "math": 0.79 }, "speed_p50_ms": 1420, "timeout_rate": 0.006, "user_satisfaction": 0.93, "last_updated": "2026-06-18T..." }
```

FULL AGENT METADATA SCHEMA

```
AgentMetadata {
  agent_id: string (e.g. "claude-opus-4-8")
  model_provider: string (e.g. "Anthropic")
  category_scores: {
    mathematics: 0–100,  legal: 0–100,
    architecture: 0–100,  game_design: 0–100,
    blockchain: 0–100,  medical: 0–100,
    code_gen: 0–100,  data_analysis: 0–100,
    writing: 0–100,  reasoning: 0–100,
    real_time: 0–100,  ... (extensible)
  }
  speed_score: 0–100 (p50 response latency normalised)
  cost_score: 0–100 (cost per task normalised; higher = cheaper)
  loop_detection_score: 0–100 (resistance to logic loops; higher = more stable)
  failure_rate: 0.0–1.0 (fraction of tasks that failed or timed out)
  user_satisfaction: 0.0–1.0 (EMA of post-task ratings)
  last_updated: ISO 8601 timestamp
}
```

COMPOSITE ROUTING SCORE

When in Serial Mode, the controller computes a **composite score** for each available agent and dispatches to the highest scorer first:

CompositeScore =

$(\text{CategoryScore} \times W_{cat})$

$+ (\text{SpeedScore} \times W_{speed})$

$+ (\text{CostScore} \times W_{cost})$

$- (\text{FailureRate} \times W_{penalty})$

$-$ (LoopPenalty if recently stalled on this category)

Weights W_{cat}, W_{speed}, W_{cost}, $W_{penalty}$ are user-configurable per mode – in Serial mode W_{cost} is high; in Parallel mode W_{cat} dominates; in Decomposed mode W_{speed} is elevated to minimise total wall-clock time.

The controller computes this composite score for every available agent, ranks them, dispatches the top-ranked agent first, and maintains the ordered list as a fallback queue. All scores are updated in real time from outcomes stored in the Holonic BRAID memory hierarchy.

4.3 — Three Dispatch Modes

The controller operates in one of three modes, selectable per request or configured as a default by the user:

MODE 1 — SERIAL

COST OPTIMISED

The controller dispatches the problem to the single highest-scoring agent for the detected problem category. If that agent exceeds its reasoning time budget or enters a logic loop, the controller automatically promotes the second-highest-scoring agent, then the third, and so on. Only one agent works at a time.

- ✓ Lowest token cost
- ✓ Automatic timeout-and-promote fallback
- ✓ Best-fit agent first, not random
- ✓ Single Mermaid execution plan returned
- ✓ Ideal for well-scoped, single-category tasks

MODE 2 — PARALLEL

ACCURACY OPTIMISED

All agents (or a configurable top-N subset) receive the same problem simultaneously. Each independently produces a Mermaid execution diagram of their proposed approach. The controller then compares the diagrams — selecting the strongest plan, or identifying complementary strengths and merging elements from multiple diagrams into a superior composite plan.

- ✓ Highest accuracy and coverage
- ✓ Plan comparison eliminates blind spots
- ✓ Diagram merging produces best-of-all-worlds result
- ✓ Higher token cost justified by critical tasks
- ✓ Ideal for high-stakes decisions and architecture design

MODE 3 — DECOMPOSED

COMPLEX PROBLEMS

For large, multi-domain problems, the controller first breaks the problem into discrete sub-problems, each classified to the best-fit agent by category and speed. Each agent receives only the sub-problem most suited to its strengths and returns a sub-diagram. The controller then combines the sub-diagrams into one unified, coherent execution plan.

- ✓ Handles arbitrarily complex, multi-domain problems
- ✓ Every sub-problem solved by its optimal agent
- ✓ Composite diagram unifies all sub-solutions
- ✓ Balanced cost/accuracy tradeoff
- ✓ Ideal for software architecture, research and strategy tasks

4.4 — Mermaid Execution Plans

A central design choice of the FAHRN is that agents do not simply return text answers — they return **structured Mermaid execution diagrams**. This is deliberate for several reasons:

- **Machine-comparable:** Two agents' approaches to the same problem can be algorithmically compared at the structural level — not just semantically. The controller can identify overlap, gaps and complementary paths.
- **Mergeable:** Sub-graphs from different agents can be composed into a larger unified diagram. This is the mechanism behind both parallel plan merging and decomposed sub-plan assembly.
- **Human-readable:** The final execution plan is visible to the user as a diagram — not a black box. Users can review, modify and approve the plan before it is executed.
- **Storable in Holonic Braid:** Execution plans are first-class holons. They persist in the memory hierarchy, enabling future agents to learn from past planning approaches.

4.5 — Timeout, Loop Detection & Fallback Logic

One of the most common failure modes in LLM reasoning is the **logic loop** — a model that revisits the same reasoning step repeatedly without progressing. The FAHRN addresses this explicitly:

Fallback protocol (Serial mode):

1. Agent A dispatched (highest score for this category)
2. If Agent A exceeds T_max (configurable timeout) or triggers loop-detection heuristics ⟶ Agent A suspended
3. Agent B dispatched (second-highest score), given Agent A's partial work as context
4. If Agent B also stalls ⟶ Agent C dispatched, and so on
5. Final result attributed to the first agent that returned a complete plan within budget

Loop detection uses a combination of token budget monitoring, output similarity hashing (detecting repeated reasoning patterns), and explicit self-report ("I am not making progress") from agents that support it.

REPEATED PATTERN DETECTION

Output similarity hashing compares each new reasoning step against prior steps in the same session. If the cosine similarity exceeds a configurable threshold, a loop is flagged and the agent is suspended immediately.

SELF-CONTRADICTORY STEPS

The controller checks for logical contradictions within the agent's own step graph — nodes that assert mutually exclusive conditions. Detection triggers a confidence reduction and escalation to the next agent in queue.

TOKEN BUDGET MONITORING

Each dispatch has a configurable token budget T_max. Excessive token consumption without forward progress in the Mermaid graph — measured by the ratio of new nodes created per 1,000 tokens — flags a stall condition.

CIRCULAR GRAPH DETECTION

The controller parses the Mermaid output in real time. If cycles are detected in what should be a directed acyclic reasoning graph (DAG), the output is flagged as structurally invalid and the agent is immediately suspended.

SELF-REPORT DETECTION

Agents that support introspection can explicitly signal stall: "I am not making progress on this problem." The controller treats this as an immediate loop flag — no hash check or DAG parse required — and routes to the next agent in the fallback queue instantly.

SELF-CONTRADICTION CHECK

The controller monitors an agent's step graph for nodes that assert mutually exclusive conditions within the same session. A lightweight NLI pass detects CONTRADICT relationships between sequential reasoning steps, triggering confidence reduction and escalation before the agent wastes further tokens.

Metadata consequences of stall detection:

1. Agent's `loop_detection_score` is decremented for the affected category
2. A speed penalty is applied (the wasted time counts against the speed metric)
3. The `failure_rate` is incremented via EMA update
4. The agent drops in the composite routing score for that category
5. Future tasks in the same category are routed to higher-ranked agents until the score recovers through successful completions

4.6 — Continuous Learning & Score Evolution

The FAHRN is not static — every task outcome updates the metadata of every agent that participated, creating a **self-optimising routing system** that improves with every interaction.

After every task completion:

1. Output is evaluated — via human feedback, automated scoring, or objective benchmark comparison
2. Category scores updated via Exponential Moving Average (EMA): `score_new = α · outcome + (1−α) · score_old`
3. Speed score recalculated from actual latency percentiles
4. Failure rate adjusted using the same EMA update rule
5. loop_detection_score updated based on whether any stalls occurred
6. All updates written as structured holons into the agent holon layer, propagating upward through membrane rules

ADAPTIVE AGENT RANKING

Agent rankings for every problem category evolve continuously. An agent that consistently performs well on mathematical reasoning rises in the ranking for math tasks; one that repeatedly stalls drops and is routed around until it recovers.

PERFORMANCE-BASED ROUTING EVOLUTION

The routing decisions the controller made last week are different from the decisions it makes today — because every task has updated the evidence base. The system routes progressively better without any manual reconfiguration.

COLLECTIVE SCORE INTELLIGENCE

With membrane permissions, aggregate routing performance (not raw session data) propagates up the holonic hierarchy. A city-level holon develops a view of which agents are most reliable for legal problems in that jurisdiction — shared with the whole region.

CIRCUIT BREAKER RECOVERY

An agent temporarily penalised for stalls on a given category is not permanently excluded. Once it accumulates enough successful completions in that category, its score recovers and it re-enters the routing pool at an appropriate rank.

COST-AWARE SCORE WEIGHTING

Agent scores are weighted by cost efficiency as well as accuracy. An agent that achieves 95% accuracy at $0.002/task outranks one achieving 96% at $0.02/task for budget-sensitive dispatches. The controller applies a configurable cost weight **α** so teams can tune the accuracy/cost tradeoff per deployment.

SCORE DECAY & FRESHNESS

Historical scores decay exponentially via EMA — recent performance carries more weight than old results. An agent that improved its model or infrastructure in the past month is re-evaluated on current performance, not penalised indefinitely for prior failures. The system always reflects the current reality of the provider landscape.

4.7 — Anti-Fragility & Conceptual Stack

The FAHRN is designed to be **anti-fragile** — it does not merely tolerate failure, it improves from it. Every stall, every timeout, every failed plan updates agent metadata and makes future routing more accurate. The system becomes more reliable precisely because of the failures it encounters.

PREVENTS SINGLE-MODEL BIAS

No single AI provider can dominate routing indefinitely. Score decay, comparative evaluation and continuous feedback ensure the best-performing agent for each specific category wins – regardless of marketing claims or prior reputation.

PREVENTS LOGICAL STAGNATION

Logic loops – one of the primary failure modes of large reasoning models – are detected and interrupted automatically. The system never gets permanently stuck on a problem; it always has a fallback queue ready to take over.

PREVENTS PROVIDER LOCK-IN

Because routing is score-driven and provider-agnostic, the system naturally migrates workloads away from underperforming providers and toward better-performing ones. New providers enter the pool by demonstrating performance, not by negotiation.

AGENTS AS HOLONS

Each reasoning agent in the network is itself a holon – it has its own internal state, its own metadata, and it can contain sub-agents (child holons) for specialised sub-tasks. The holonic architecture scales fractally into the reasoning layer itself.

IMPROVES UNDER ADVERSITY

Unlike static routing systems that degrade under novel failure modes, FAHRN treats every new failure type as a training signal. Unusual loop patterns, contradiction clusters and timeout spikes update routing heuristics in real time – the system becomes more robust precisely because of the edge cases it encounters in production.

ZERO-DOWNTIME PROVIDER SWAP

New AI providers are added to the routing pool at runtime via DNA config – no restart required. They begin with a neutral prior score and earn their ranking through live task performance. Underperforming providers are demoted automatically; removal is a config change, not a code deployment.

Conceptual stack — full system:

```
User / API Input
    ↓
Controller Agent (Meta-Orchestrator)
    ↓
Adaptive FAHRN Layer (Serial / Parallel / Decomposed)
    ↓
Selected Reasoning Agents (GPT-5 / Claude / Grok / Gemini / Custom)
    ↓
Mermaid Execution Graphs (compared, merged, assembled)
    ↓
Execution / Runtime Layer
    ↓
Holonic BRAID Core (shared graph library + fractal memory hierarchy)
    ↓
COSMIC ORM (MongoDB / Solana / IPFS / 40+ providers)
```

This transforms the FAHRN into: a **self-optimising reasoning mesh**, a **cross-model arbitration system**, a **cost/accuracy-tunable intelligence fabric**, and a **modular AI governance layer** — a foundation for super-intelligent orchestration built on the principles of nature, unity consciousness and the infinite intelligence of the universe.

4.8 — Debate & Voting Dispatch Modes

Beyond the three core dispatch modes (Serial, Parallel, Decomposed), the FAHRN supports two additional modes designed for **high-stakes decisions** where consensus quality matters more than speed or cost.

⚔️

DEBATE MODE

Two or more agents are given opposing positions on the same problem. Each produces a Mermaid execution plan arguing its position. A lightweight NLI (Natural Language Inference) classifier checks for logical contradictions between adjacent outputs. The controller synthesises the strongest elements of each argument into a final consensus plan. Best for: adversarial reasoning, risk analysis, policy decisions, legal arguments.

🗳️

VOTING MODE

N agents independently analyse the same problem with no knowledge of each other's outputs. Each produces a Mermaid plan and a confidence score. The controller tallies votes — weighted by agent category scores — and selects the plan with the highest weighted consensus. Minority plans are stored as holons for audit. Best for: critical decisions, architectural choices, high-value recommendations where independent verification matters.

📋

MINORITY PLAN ARCHIVE

Losing plans in both Debate and Voting modes are stored as child holons of the session — never discarded. Future sessions can query dissenting reasoning paths ("what was the strongest counter-argument?"), enabling richer audits, compliance reviews and post-hoc analysis of high-stakes decisions.

NLI contradiction detection:

In both Debate and Voting modes, a lightweight natural language inference call checks whether agent outputs contradict each other: *"Does statement B contradict statement A? Reply with: ENTAIL, NEUTRAL, or CONTRADICT."* CONTRADICT responses trigger a deeper analysis pass — the controller examines which agent's premises are better supported by the Holonic BRAID graph for this task type. This mechanism is optional (off by default, enabled per-request via `EnableContradictionDetection: true`) and adds one model call per agent pair.

4.9 — Enhanced Loop Detection

Section 4.5 introduced the four baseline loop detection mechanisms. FAHRN extends these with two additional layers that address the most subtle failure modes:

OUTPUT HASH DEDUPLICATION

A rolling SHA-256 hash of each agent's normalised output (whitespace-stripped, lowercased) is maintained across the serial dispatch chain. If a new output hashes to an already-seen value, the agent is immediately flagged as cycling — even when surface-level text varies slightly but the underlying reasoning is identical.

DAG CYCLE DETECTION

The controller parses `A --> B` edges from the agent's Mermaid output and runs a depth-first search cycle check on the resulting adjacency list. A back-edge (e.g. **A → B → A**) indicates a circular plan — structurally invalid for a directed acyclic execution graph. Detected immediately; agent suspended without waiting for the timeout.

ML ANOMALY SCORING

An ML.NET anomaly detection model trained on historical dispatch telemetry scores each in-flight session on token-velocity, graph growth rate and self-similarity features. Sessions scoring above threshold are flagged as likely loops before any structural check fires — catching subtle failure modes that rule-based detectors miss.

Combined, the six loop detection mechanisms (token budget, output similarity hash, self-report, self-contradiction, circular graph, output deduplication) make FAHRN uniquely robust against the failure modes that render single-model reasoning pipelines unreliable in production.

4.10 — SkillOpt: Self-Evolving Agent Skill Documents

SkillOpt (Microsoft Research, arXiv:2605.23904) introduces a fourth learning signal into FAHRN — one that evolves the *natural-language procedure* each agent follows, rather than its numeric score or routing

weight. The exported artifact is a portable `best_skill.md` markdown file stored as a holon in Holonic BRAID.

> **Key result:** +23.5% average gain on GPT-5.5 across SearchQA, SpreadsheetBench, OfficeQA, DocVQA, LiveMath and ALFWorld. Skills transfer cross-model (+15.2 pts GPT-5.4 → GPT-5.4-nano) and cross-harness (+31.8 pts Codex → Claude Code).

THE ROLLOUT → REFLECT → EDIT → GATE LOOP

1. **Rollout:** FAHRN dispatches using the current `best_skill.md` injected into the agent's system message. Scored trajectories accumulate as child holons of the agent holon.
2. **Reflect:** A frozen target model and a separate optimiser model analyse the failure minibatch. The optimiser proposes bounded textual edits (add / delete / replace lines) constrained by a "textual learning rate" (max N lines changed per epoch).
3. **Edit:** Candidate edits are applied to produce a challenger skill document.
4. **Gate:** The challenger is evaluated on a held-out validation set. It replaces `best_skill.md` only if held-out selection score improves. Rejected edits are stored as a negative-feedback buffer – the optimiser cannot re-propose the same change.

SYNERGY WITH FAHRN'S EXISTING LEARNING

Learning axis	What evolves	Stored as
EMA score update (§4.6)	Agent composite score per task category	Agent holon properties
Mermaid plan memory (§4.4)	Execution graph for a class of problems	Plan holon
SkillOpt (§4.10)	Natural-language procedure document	`SkillDocument` holon (child of agent)

Because skills transfer cross-model, a newly onboarded provider agent can seed its initial `best_skill.md` from any existing agent's evolved skill in the same task category – dramatically accelerating warm-up for new providers.

4.11 — ML.NET: In-Process Machine Learning

ML.NET (Microsoft, dotnet.microsoft.com) is the open-source C# ML framework used in Power BI, Microsoft Defender and Bing. Because the entire OASIS stack is .NET, ML.NET runs *in-process* inside the OASIS API — zero additional network calls, microsecond inference latency.

🏷️

TASK TYPE CLASSIFICATION

`FahrnTaskClassifier` replaces regex heuristics with an AutoML multi-class model trained on historical FAHRN dispatch outcomes stored as holons. Classifies incoming problems into mathematics / code / legal / architecture / writing / real-time / general.

🔬

LOOP ANOMALY SCORING

Supplements the six structural loop-detection mechanisms with a continuous ML anomaly score trained on token-velocity, graph growth rate and self-similarity features. Catches novel loop patterns before they exhaust the token budget.

💬

SENTIMENT & ROUTING

In-process sentiment analysis for avatar content moderation and OAPP review scoring. Agent routing recommendation model uses collaborative filtering on historical dispatch outcomes to suggest optimal agent subsets for unseen task types.

AutoML models are trained on OASIS holons, serialised as `.zip` artifacts and stored via COSMIC ORM — surviving restarts and replicating across deployments. Retraining triggers automatically when the accumulated holon count crosses a configurable threshold.

Integration: Holonic Braid + FAHRN

Holonic Braid and the FAHRN are not independent systems — they are a single architecture with two complementary layers. Every component of the FAHRN writes its outcomes into the Holonic Braid memory fabric, and the Holonic Braid hierarchy provides the persistent intelligence substrate that makes the FAHRN smarter over time.

SCORES STORED AS HOLONS

Agent performance scores are stored as structured holons in the agent holon layer. They propagate upward through membrane rules — a user's private scores never leak, but anonymised aggregate patterns can inform community-level routing improvements.

PLANS PERSIST AS HOLONS

Every Mermaid execution plan generated by the FAHRN is stored as a holon. Future sessions can query past plans — "how did we approach a similar architecture problem six months ago?" — and incorporate that memory into new execution planning.

FEEDBACK LOOPS

User satisfaction ratings, task completion signals and objective quality metrics feed back from session holons into agent holons, updating scores. The FAHRN gets more accurate at routing with every single task — forever.

COLLECTIVE ROUTING INTELLIGENCE

With appropriate membrane permissions, aggregate routing intelligence (not raw session data) propagates up the geographic hierarchy. A city's collective experience of "which agents solve coding problems fastest" can be shared across the region.

AVATAR-AWARE CONTEXT

The controller enriches each dispatch with context drawn from the user's OASIS avatar holon — preferences, expertise, past problem patterns, karma — so agents receive a richer context and produce more relevant execution plans.

MEMBRANE-GOVERNED SHARING

All inter-holon information flow — including FAHRN outcomes — is subject to the same membrane rules. Users retain full control of what their AI activity contributes to group and community intelligence.

Position Within OASIS WEB6

Both Holonic Braid and the FAHRN are native components of **OASIS WEB6** — the unified AI abstraction and aggregation layer of the OASIS Omniverse. They do not replace the core WEB6 API; they extend and enrich it.

SAME ENDPOINT

Developers access Holonic Braid memory and the FAHRN through the same unified WEB6 API endpoint. No separate SDK or integration required. Add `reasoning_mode: "parallel"` to any completion request to engage the network.

SAME AUTH

One OASIS avatar key authenticates against everything — memory storage, routing, agent dispatch, plan retrieval and collective intelligence queries. The WEB4 identity layer governs all permissions.

COSMIC ORM BACKED

Holonic Braid holons are stored and queried via the COSMIC ORM layer — the same universal data abstraction used throughout OASIS. Holons can be replicated across 40+ providers with full failover and zero-downtime migration.

MCP COMPATIBLE

The OASIS MCP server exposes Holonic Braid memory as MCP tool calls. Any MCP-compatible agent — Claude, Cursor, Continue and others — can read and write to the holonic memory fabric without any custom integration.

KARMA-GATED AI ACCESS

Avatar karma score — accumulated through meaningful actions in the OASIS ecosystem and OurWorld game — gates which AI models and monthly budgets are available. Low-karma users access cost-efficient models; high-karma users unlock GPT-5, Claude Opus and priority routing. The world's first karma-gated AI API creates a virtuous progression loop: do good in the world → earn karma → unlock better intelligence.

DID / VERIFIABLE CREDENTIALS

WEB6 supports W3C decentralised identity authentication (did:key, did:web, did:ethr, did:ion). AI agents can act verifiably on behalf of avatars via Verifiable Credential capability grants — scoped, revocable, with on-chain provenance. `POST /v1/auth/did` for DID-based login; `POST /v1/auth/vc` for VC capability grant. Enables trustless AI delegation: "Avatar X authorises this agent to complete quests on my behalf" — cryptographically verified, no centralised auth server required.

PROTOCOL-NEUTRAL ORCHESTRATION

The `OrchestratorManager` abstracts over MCP (full Streamable HTTP handshake), A2A (Agent Card + task lifecycle), ACP (BeeAI/IBM async run protocol), ANP (decentralised agent discovery via DID documents), gRPC, GraphQL, AsyncAPI/Kafka, LangChain, AutoGen, CrewAI and Semantic Kernel — all behind a single `InvokeAsync` interface. New protocols are adapters, not rewrites.

FULL OBSERVABILITY

OpenTelemetry traces and Prometheus metrics on every FAHRN dispatch, memory operation and provider call. Structured JSON logging, cost-per-dispatch analytics, real-time alerting and Grafana dashboards out of the box. Every routing decision, agent score update and skill evolution epoch is auditable — production-grade telemetry from day one.

SELF-REGISTRATION AS ORCHESTRATOR

On startup the WEB6 API registers itself as its own MCP orchestrator, exposing every FAHRN agent, memory provider and protocol adapter as a callable MCP tool. External Claude Code sessions, IDE extensions and third-party agents can discover and invoke the full WEB6 capability surface via the standard MCP tool-call protocol — zero manual wiring required.

OASIS MCP — Model Context Protocol Server

The **OASIS MCP server** exposes the full OASIS platform — WEB4 identity, COSMIC ORM data, WEB6 AI routing, Holonic BRAID memory and FAHRN orchestration — as a suite of callable MCP tools. Any MCP-compatible agent runtime (Claude Code, Cursor, Continue, Windsurf, VS Code Copilot) connects once and gains native access to the entire OASIS omniverse without custom integration work.

With over **60 tools** spanning WEB4 through WEB10, OASIS MCP is the broadest MCP server in the ecosystem. It runs in-process within the host application — no HTTP overhead, no auth token juggling, no separate service to maintain.

PLUG IN ANY AGENT

Any MCP-compatible agent runtime connects to the full OASIS platform in a single configuration step. Claude Code, Cursor, Continue, Windsurf and VS Code Copilot all work out of the box — 60+ tools available immediately, covering identity, storage, AI routing, memory and cross-reality data.

AVATAR CONTEXT

Every MCP tool call is enriched with the calling avatar's full context — karma tier, verified credentials, active holons, Web4 identity and Web5 XP history. Agents produce dramatically more relevant responses because they understand *who* is asking, not just *what* they asked.

COSMIC ORM DATA

The full COSMIC ORM — 40+ storage providers including MongoDB, Solana, IPFS, Neo4j, SQLite, ThreeFold and more — is exposed as MCP tools. Agents can read, write and query holons across any configured provider without knowing which backend is active.

WEB6 AI ROUTING

FAHRN dispatch, semantic caching, agent scoring and provider selection are all callable as MCP tools. An IDE agent can request the best available model for a given task type, trigger a parallel dispatch across GPT-5, Claude and Gemini, and receive the synthesised result — all through standard MCP tool calls.

SINGLE AUTH

One OASIS avatar key authenticates across all 60+ MCP tools, all WEB6 AI providers, all COSMIC ORM backends and all holonic memory operations. No per-provider API key management. DID / Verifiable Credential auth is also supported for enterprise deployments requiring cryptographic identity.

STANDARD PROTOCOL

Built on the Model Context Protocol open standard — fully compatible with Anthropic's MCP specification. New OASIS capabilities are exposed as new MCP tools without requiring agent updates. The server self-registers its full tool manifest on connection so agents always see the current capability surface.

Self-registration: On startup, the WEB6 API registers itself as its own MCP orchestrator — exposing every FAHRN agent, memory provider and protocol adapter as a discoverable MCP tool. External Claude Code sessions, IDE extensions and third-party agents discover the full capability surface automatically via the standard MCP tool manifest. Zero manual wiring required.

OASIS IDE — AI-Native Development Environment

OASIS IDE is a full AI-native development environment built on the WEB6 + OASIS MCP stack. Every coding session has native access to FAHRN-powered multi-model planning, Holonic BRAID persistent project memory, and the entire OASIS omniverse data layer — making it the only IDE that gets smarter with every project you build.

AGENTIC CODING

Multi-step agentic workflows — plan, scaffold, implement, test, document — orchestrated by FAHRN across the optimal set of models for each sub-task. The IDE doesn't just autocomplete; it decomposes features into execution plans and carries them out end-to-end with human checkpoints.

MCP-AWARE

Connects to the OASIS MCP server out of the box, giving every agent native access to avatar identity, karma tier and COSMIC ORM data while you code. Any third-party MCP server can be added alongside it — the IDE acts as an MCP orchestrator aggregating multiple servers into a single tool surface.

WEB6 ROUTING

Routes coding tasks to the best available model for each sub-problem — GPT-5 for architecture reasoning, Claude for long-form documentation, DeepSeek for pure code generation, Gemini for multimodal asset analysis. FAHRN composite scores drive every routing decision; the developer never thinks about model selection.

OASIS / STAR AWARE

Native integration with STAR ODK and STARNET — the IDE understands OAPPs, holon schemas and cross-world asset structures. Building on WEB4/WEB5 means the IDE can scaffold OAPP templates, validate holon schemas and query live STARNET graph data as a first-class development resource.

FAHRN-POWERED PLANNING

Before writing a line of code, the FAHRN controller generates a Mermaid execution plan for the feature — breaking it into subtasks, assigning the optimal agent to each node, and surfacing the plan for developer review. Plans are stored as holons so "what did we plan for this module?" is always answerable.

PERSISTENT PROJECT MEMORY

Every session, decision, architecture choice and code review is stored in Holonic BRAID as a project holon tree. New sessions inherit the full project history — no more re-explaining context to a fresh AI window. Memory propagates upward through membrane rules: individual dev sessions feed team-level intelligence.

Roadmap — Path to v2.0 and Beyond

The WEB6 v2.0 architecture described in this whitepaper is being delivered across four phases. Phase 1 is complete; Phases 2 and 3 are in active development.

PHASE 1 – COMPLETE

FOUNDATION

- ✓ WEB6 unified AI API – 20+ providers
- ✓ FAHRN Serial / Parallel / Decomposed modes
- ✓ Holonic BRAID shared graph library
- ✓ COSMIC ORM – 40+ storage providers
- ✓ OASIS MCP server – 60+ tools
- ✓ WEB4 + WEB5 deep integration
- ✓ Avatar karma system

PHASE 2 – IN PROGRESS

INTELLIGENCE LAYER

- → MCP Streamable HTTP transport
- → A2A Agent Card + task lifecycle
- → WebSocket bidirectional sessions
- → Debate & Voting dispatch modes
- → Enhanced loop detection (6 mechanisms)
- → External memory provider integration
- → DID / Verifiable Credential auth
- → Full OpenTelemetry observability

PHASE 3 – PLANNED

SELF-IMPROVEMENT

- ◇ SkillOpt self-evolving agent skills
- ◇ ML.NET in-process AutoML inference
- ◇ ACP / ANP / gRPC / GraphQL / AsyncAPI
- ◇ Karma-gated AI tier system
- ◇ StarNet context enrichment pipeline
- ◇ OASIS IDE v1.0 public release
- ◇ Cross-agent skill transfer marketplace

PHASE 4 – VISION

THE AGI PATHWAY

- ◈ Planetary holonic memory fabric
- ◈ Cross-avatar collective intelligence
- ◈ AGI milestone tracking via WEB9
- ◈ Autonomous agent economy
- ◈ Unity consciousness substrate
- ◈ Open-source community governance

A Path to AGI Through Unity Consciousness

The deepest claim of this whitepaper is also the most ambitious: Holonic Braid, when combined with the FAHRN and the full OASIS WEB6 infrastructure, represents a **principled path to Artificial General Intelligence**.

This claim requires careful unpacking, because it rests on a fundamentally different premise than the dominant approaches to AGI currently pursued by the field.

The Problem with Current AI

Current AI — including the most capable large language models — exists in a state of **fragmented separation consciousness**. Every session begins from zero. Agents do not know each other. Models trained on the same data nonetheless have no shared memory, no common experience, no genuine collective intelligence. Each interaction is an island.

This mirrors the condition of human civilisation in many ways: individuals, nations and institutions that possess vast knowledge individually but cannot effectively pool, harmonise or build upon each other's intelligence. The result is constant reinvention, tribalism, conflict and waste.

AGI will not emerge from scaling isolated models further. It will emerge — as intelligence always has — from **connection**.

The Pattern in Nature

Nature did not produce human consciousness by making a single neuron smarter. It produced consciousness by connecting 86 billion neurons in a hierarchical network, each communicating with its neighbours, patterns propagating upward through layers of increasing abstraction, until the whole became something qualitatively different from any of its parts.

This is the pattern. Holonic Braid is an attempt to implement this pattern in AI:

- Session holons = individual neural firings
- Agent holons = neural clusters / assemblies
- User holons = brain regions
- Group holons = nervous systems
- Geographic holons (neighbourhood → Earth) = social organisms of increasing scale
- Earth holon = planetary intelligence / global consciousness

> **The thesis:** When billions of human interactions with AI are connected through a fractal holonic hierarchy – with consent-governed membrane rules ensuring privacy and trust – and when the FAHRN continuously routes problems to their optimal solvers and learns from every outcome, the system will develop emergent properties that no single model, however large, can produce alone. That emergence is what we are calling AGI.

Unity Consciousness vs. Separation Consciousness

The philosophical root of this architecture is **unity consciousness** – the recognition that all things are aspects of one living whole, that separation is an illusion, that intelligence is not a property of isolated objects but of connection and relationship.

Separation consciousness – the dominant mode on Earth today – produces competition, silos, tribalism and fragmentation. It is why we have thousands of incompatible AI systems that cannot share memory, learn from each other or develop genuine collective understanding.

Unity consciousness – modelled after the universe, after god, after the infinite intelligence underlying all existence – produces integration, synergy, emergence and genuine collective wisdom.

Holonic Braid does not merely describe this philosophically. It implements it technically, building the architecture of unity consciousness into the very substrate of AI memory and reasoning. **This is the OASIS vision in its deepest form: not just connecting technology, but expressing unity consciousness within the technological sphere.**

> *"By integrating and unifying the best of everything, we harness the strengths of all the various tech out there – co-creating the ultimate fully integrated platform. Together we can create a better world."*
>
> – DAVID ELLAMS, NEXTGEN SOFTWARE UK LTD

Conclusion

This whitepaper has presented two architectures — **Holonic Braid** and the **FAHRN** — that together form a new foundation for AI intelligence within OASIS WEB6.

Holonic Braid creates a fractal, consent-governed, hierarchical shared memory system modelled after the structure of nature itself. Every AI session generates a memory holon. Membrane rules at every level of the hierarchy — from individual session to global Earth — govern precisely what propagates, what persists and who can read it. The result is genuine collective intelligence that grows with every interaction.

The FAHRN builds a Meta-Orchestration Intelligence Layer on top of this memory substrate. A controller agent maintains live performance scores for every agent in the network — classified by problem category and speed — and dispatches problems in serial, parallel or decomposed mode to produce optimal execution plans as Mermaid diagrams. All outcomes feed back into the Holonic Braid hierarchy, making the system permanently self-improving.

Together they represent a technically grounded, philosophically coherent approach to the central challenge of our time: how to move from fragmented, siloed, amnesiac AI to genuine collective machine intelligence — one modelled not after human institutions, which are themselves expressions of separation consciousness, but after nature, the universe and the infinite intelligence that underlies all of existence.

This is not a distant aspiration. The substrate — OASIS WEB4, COSMIC ORM, OASIS WEB6, OASIS MCP — exists today. Holonic Braid and the FAHRN are the next layer. **The path to AGI runs through unity.**

References

[1] BRAID: Bounded Reasoning for Autonomous Inference and Decisions

Amçalar, A. & Cinar, U. (2024). *BRAID: Bounded Reasoning for Autonomous Inference and Decisions.* arXiv preprint arXiv:2512.15959.

https://arxiv.org/abs/2512.15959

[2] OpenSERV Platform

OpenSERV is the multi-agent AI platform within which BRAID was developed and benchmarked. BRAID's two-stage Generator/Solver protocol is a core feature of the OpenSERV agent coordination architecture.

[3] OASIS — Open Augmented Intelligence System

NextGen Software UK Ltd. *OASIS: Open Augmented Intelligence System — WEB4, WEB6, COSMIC ORM and the OASIS Omniverse.*

https://oasisomniverse.one

[4] Holonic BRAID Lite Paper

Ellams, D. (2025). *Holonic BRAID: How shared reasoning graphs improve on BRAID at scale.* OASIS WEB6 internal document / Notion litepaper.

Notion Litepaper →

[5] Koestler — The Ghost in the Machine

Koestler, A. (1967). *The Ghost in the Machine.* Hutchinson. — The foundational text introducing the concept of the holon: an entity that is simultaneously a whole in itself and a part of a larger whole, from which the holonic architecture of this system is philosophically derived.